

Class-mode / EduAI

System Design

Subsystem Deep Dives

Multi-tenant AI learning platform — React 18 · Express/TypeScript · PostgreSQL · Cassandra · Redis · Gemini/OpenAI · Daily.co · GCP Cloud Run

Production: classmode.inmodel.in | Version 1.8.4.0 | Generated 2026-07-10

Contents

- 1. Auth, Identity & Multi-Tenancy (RBAC)
- 2. AI Tutor Engine
- 3. Study Arena (AI Classroom)
- 4. MessagePal Real-time Messaging
- 5. Billing & Quota
- 6. LMS & SIS Integrations

Each chapter follows the same shape: purpose, an architecture diagram, key components, the request/data flow, the data model, external dependencies, and failure modes & resilience. Editable Excalidraw source for every diagram ships alongside this PDF.

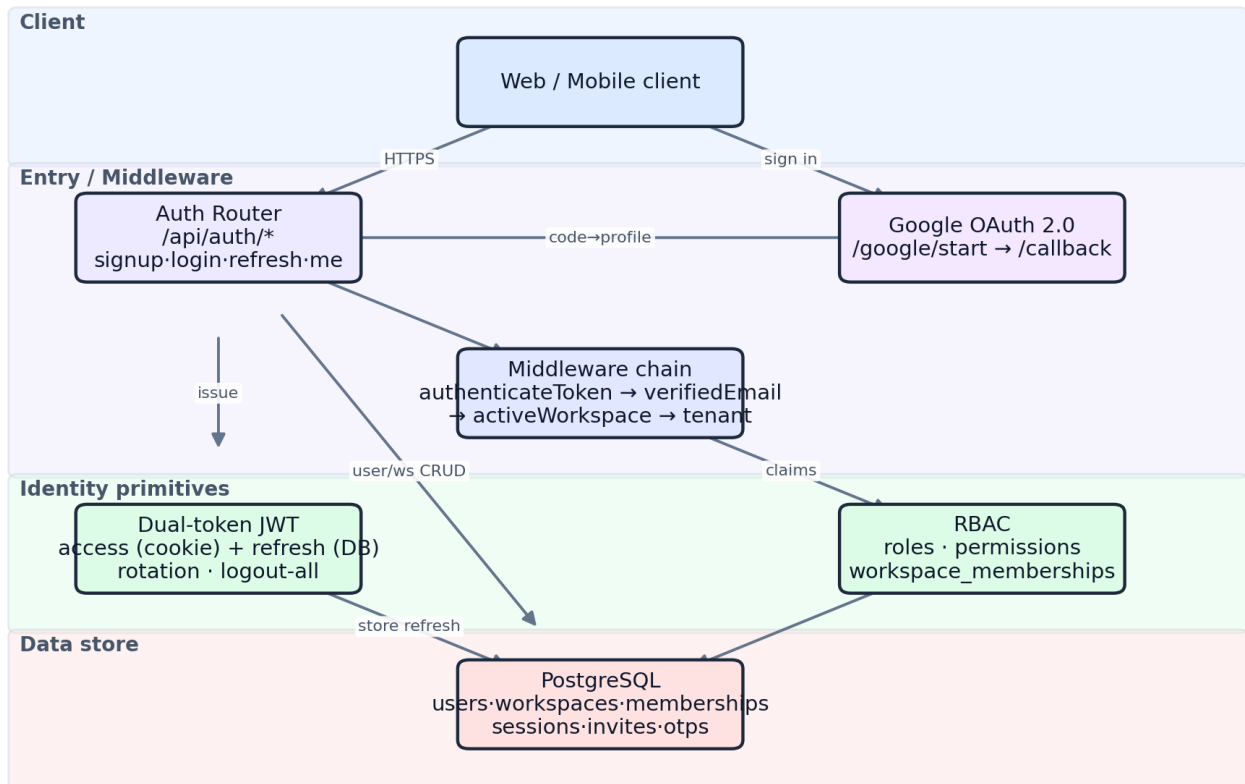
1. Auth, Identity & Multi-Tenancy (RBAC)

Purpose

Self-hosted identity with no Firebase on the hot path. Every request is authenticated, attributed to a user, and scoped to exactly one workspace (tenant) before any domain logic runs. This subsystem owns sign-up, login, Google OAuth, session lifecycle, and role-based access control.

Architecture

Subsystem 1 — Auth, Identity & Multi-Tenancy (RBAC)



Editable source: `diagrams/subsystems/01-auth-multitenancy.excalidraw`

Key components

- **Auth Router** (`server/routes/auth.ts`) — ~20 endpoints: `signup`, `workspace-invite signup`, `login`, `refresh`, `me`, `logout` / `logout-all`, `email verify request/confirm`, `password forgot/reset`, `invite validate/accept`.
- **Google OAuth 2.0** — server-side flow (`/google/start` → `/google/callback`) that dodges popup blockers and mobile in-app browsers.
- **Middleware chain** (`server/middleware.ts`) — `authenticateToken`, `requireVerifiedEmail`, `requireActiveWorkspace`, `requireRole(...)`, and `tenant scoping`.
- **Dual-token JWT** — short-lived access token (cookie) + refresh token persisted in Postgres with rotation; `logout-all` revokes every session.
- **RBAC** (`shared/authz.ts`) — user roles (`student`, `teacher`, `principal`, `school_admin`, `admin`, `parent`), statuses, and per-workspace membership roles (`owner` / `admin` / `member`).

Request / data flow

1. Client calls a protected `/api` route; `authenticateToken` verifies the access-token cookie and attaches the user.
2. If the token is expired, the client hits `/api/auth/refresh`; the refresh token is validated against the sessions table and rotated.
3. `requireVerifiedEmail` and `requireActiveWorkspace` gate access; the tenant middleware pins the request to one workspace.
4. RBAC resolves the caller's membership role + permissions; the handler runs only if the role satisfies the route.
5. Google sign-in: `/google/start` redirects to Google, `/google/callback` exchanges the code for a profile, upserts the user, and issues the token pair.

Data model

users	identity, global role, status, verified flag
workspaces	tenant root; slug, owner
workspace_memberships	user↔workspace with membership role
roles / permissions / role_permissions	granular RBAC grants
sessions	refresh tokens, rotation, revocation
invites / workspace_invites	secure onboarding tokens
otps	email verification & password reset codes

External dependencies

- Google OAuth 2.0 (identity)
- Mailer (verification & reset emails)

Failure modes & resilience

- **Token theft / replay** → short access TTL + refresh rotation; logout - all mass-revokes.
- **Cross-tenant leakage** → mandatory tenant middleware scopes every query to the active workspace.
- **Brute force** → per-route rate limiters (`loginLimiter`, `signupLimiter`, `verify/forgot` limiters).
- **OAuth failure** → callback errors redirect to a safe error route rather than 500-ing.

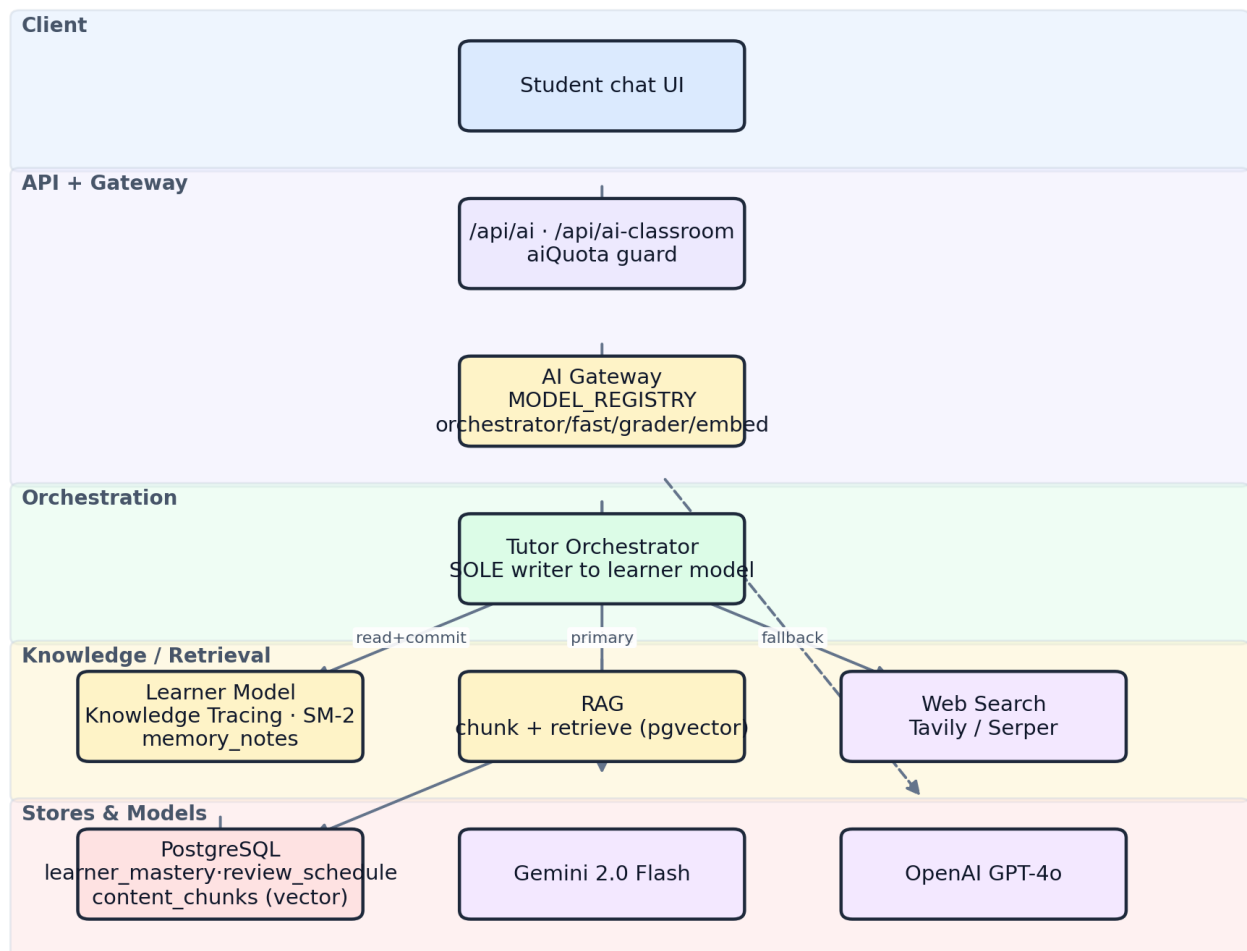
2. AI Tutor Engine

Purpose

Subject-aware tutoring that adapts to each learner. A single orchestrator composes curriculum context, the learner's mastery state, and retrieved knowledge into one coherent turn — and is the *sole writer* to the learner model, so mastery state never races.

Architecture

Subsystem 2 — AI Tutor Engine (Gateway · Orchestrator · Learner Model)



Editable source: `diagrams/subsystems/02-ai-tutor-engine.excalidraw`

Key components

- **AI Gateway** (`server/lib/gateway.ts`) — a `MODEL_REGISTRY` with named roles (`orchestrator / fast / grader / embed`) and Gemini→OpenAI fallback.
- **Tutor Orchestrator** (`server/lib/orchestrator.ts`) — per turn: read learner snapshot, inject curriculum + learner context, generate (stream), grade, then commit the outcome through the single writer.
- **Learner Model** (`learner-model.ts`, `knowledge-tracing.ts`, `spaced-repetition.ts`) — Knowledge Tracing mastery vector, SM-2 review scheduling, and durable memory notes.
- **RAG** (`server/lib/rag/`) — chunk + retrieve over pgvector-backed `content_chunks`.
- **aiQuota guard** — tiered monthly limits enforced before any model call (see Billing & Quota).

Request / data flow

1. Student message hits `/api/ai`; `aiQuota` confirms remaining budget.
2. Orchestrator loads the learner snapshot (mastery, due reviews, memory) and retrieves curriculum + RAG context.
3. Composed system + messages go through the Gateway to Gemini 2.0 Flash (OpenAI GPT-4o on failure); the reply streams back.
4. The turn is graded; `commitTurnOutcome` updates Knowledge Tracing mastery and SM-2 schedule via the single writer.
5. Optional live web search enriches answers when curriculum/RAG context is insufficient.

Data model

learner_mastery	per-skill mastery (Knowledge Tracing)
review_schedule	SM-2 spaced-repetition due dates
memory_notes	durable learner memory
content_chunks	embedded curriculum chunks (pgvector)
competencies / interaction_log	skills graph & turn history

External dependencies

- Gemini 2.0 Flash (primary)
- OpenAI GPT-4o (fallback)
- Web Search: Tavily / Serper

Failure modes & resilience

- **Primary model outage** → Gateway falls back to OpenAI transparently.
- **Concurrent writes to mastery** → orchestrator is the sole writer, eliminating races.
- **Quota exhaustion** → request rejected at the guard before spend.
- **Retrieval miss** → graceful degrade to base model + optional web search.

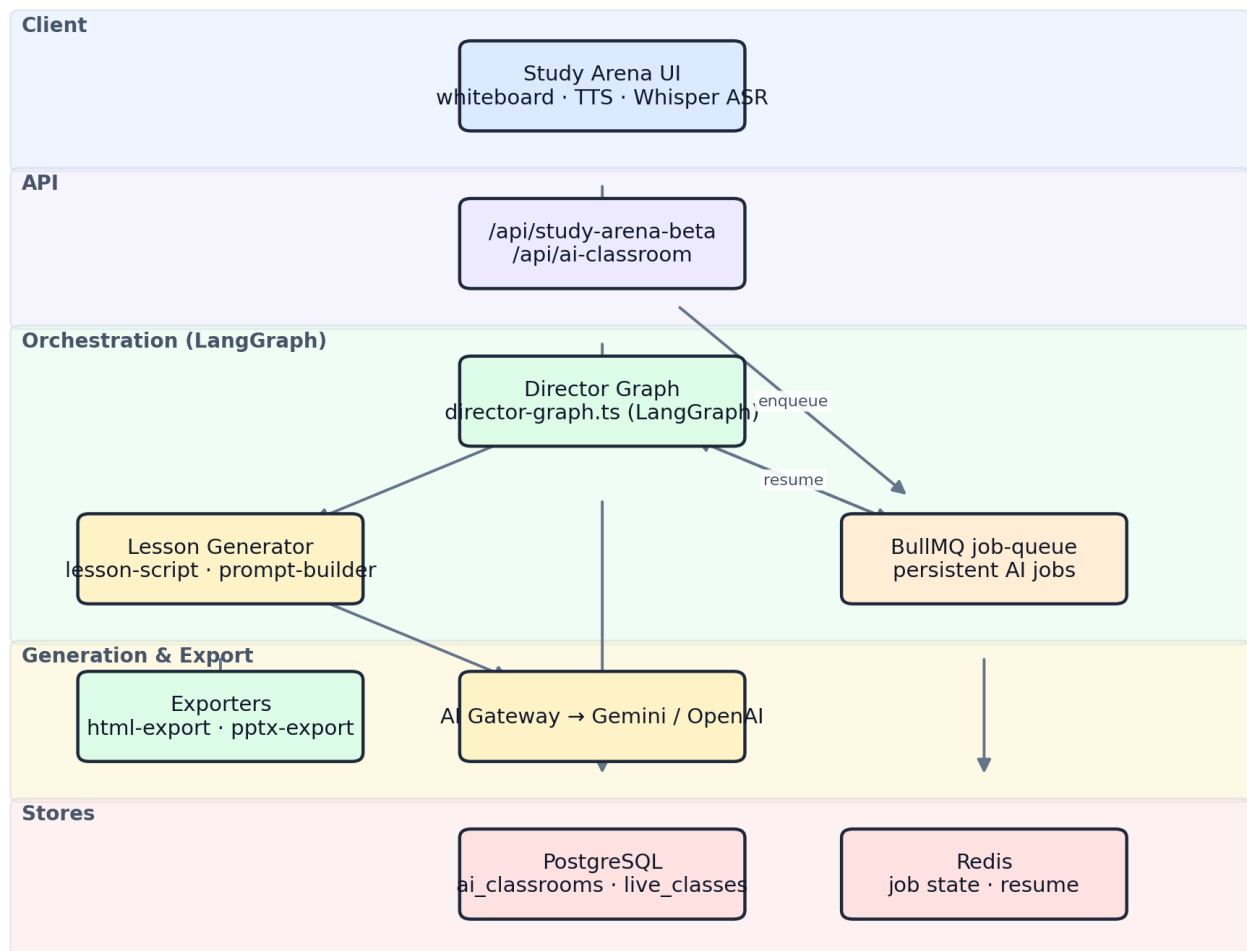
3. Study Arena (AI Classroom)

Purpose

Real-time, interactive lesson playback — whiteboard, text-to-speech, Whisper ASR, and project-based learning. A LangGraph 'director' orchestrates multi-step lesson generation as durable, resumable jobs so a lesson survives restarts and disconnects.

Architecture

Subsystem 3 — Study Arena (AI Classroom · LangGraph Director)



Editable source: `diagrams/subsystems/03-study-arena.excalidraw`

Key components

- **Director Graph** (`server/services/study-arena/director-graph.ts`) — LangGraph state machine driving lesson generation and playback.
- **Lesson Generator** (`generator.ts`, `lesson-script.ts`, `prompt-builder.ts`) — builds the structured lesson script.
- **Job Queue** (`job-queue.ts`, BullMQ) — Redis-backed persistent AI jobs with resume.
- **Exporters** (`html-export.ts`, `pptx-export.ts`) — render lessons to shareable HTML / PPTX.
- **AI adapters** (`gemini-adapter.ts`, `ai-sdk-adapter.ts`) — model access through the shared Gateway.

Request / data flow

1. Client opens a lesson via `/api/study-arena-beta / /api/ai-classroom`.
2. The route enqueues a generation job and starts the Director Graph.
3. The Director calls the Generator step-by-step; each step's model call runs through the Gateway.
4. Job state is checkpointed to Redis; on disconnect/restart the job resumes from its last node.
5. Completed lessons persist to Postgres and can be exported to HTML / PPTX.

Data model

ai_classrooms	classroom / lesson records
live_classes	scheduled live sessions
Redis (BullMQ)	job state, checkpoints, resume

External dependencies

- Gemini / OpenAI via AI Gateway
- TTS + Whisper ASR (playback)

Failure modes & resilience

- **Worker crash mid-lesson** → BullMQ + Redis checkpoint lets the job resume, not restart.
- **Long generation** → async job model keeps the request thread free.
- **Model error on a node** → retried within the graph without discarding prior steps.

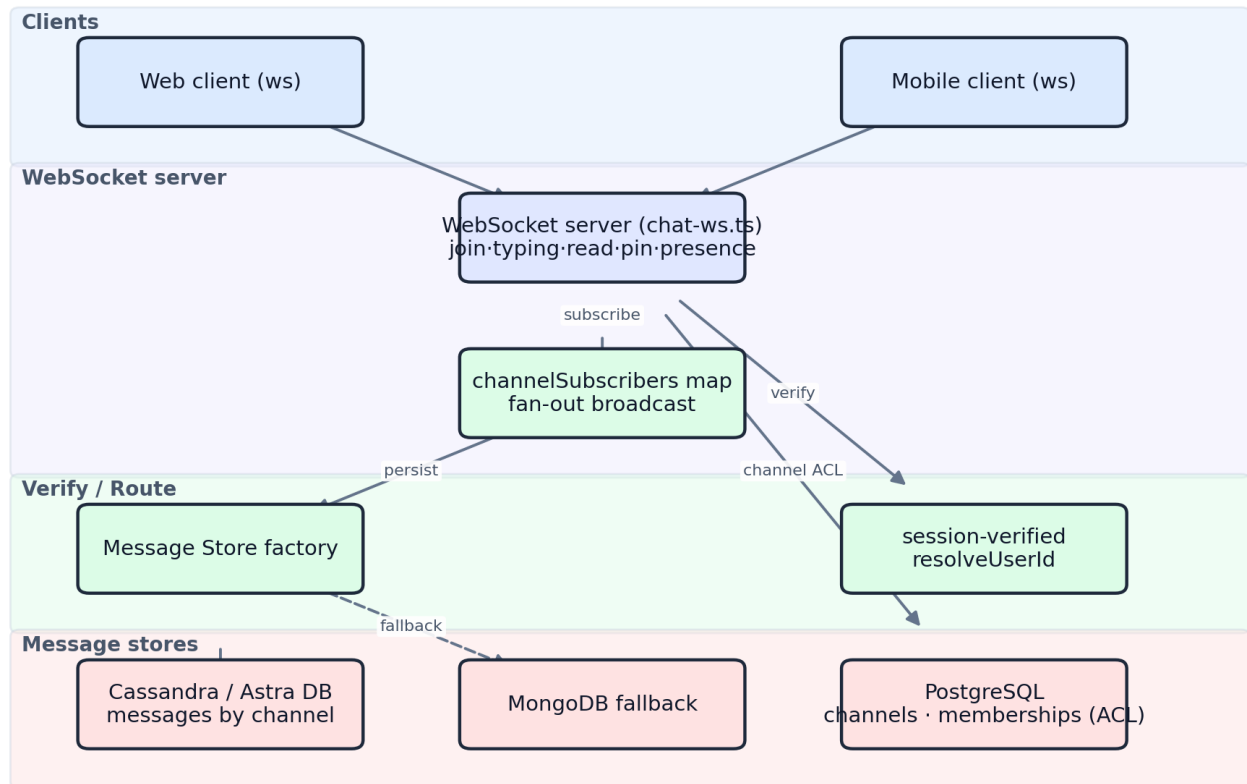
4. MessagePal Real-time Messaging

Purpose

Low-latency chat with typing indicators, read receipts, presence, and message pinning. Live delivery runs over WebSockets; durable history is stored in Cassandra (Astra DB) with a MongoDB fallback, keeping the hot chat path off the transactional Postgres.

Architecture

Subsystem 4 — MessagePal Real-time Messaging (WebSocket)



Editable source: `diagrams/subsystems/04-messagepal-realtime.excalidraw`

Key components

- **WebSocket server** (`server/chat-ws.ts`) — handles `join/leave channel`, `typing / stop_typing`, `mark_delivered`, `read receipts`, `pin_message`, `presence`.
- **Channel subscriber registry** — in-memory `channelSubscribers map (channelId → sockets)` for `O(subscribers)` fan-out.
- **Message Store factory** (`server/message/factory.ts`) — selects Cassandra or Mongo behind one interface.
- **Cassandra store** (`cassandra-message-store.ts`) — `create / query-by-channel / pinned / delete / grade / mark-read`.
- **Session verification** — `resolveUserId` ties each socket to an authenticated user.

Request / data flow

1. Client opens a WebSocket; the server verifies the session and resolves the user.

2. Client sends `join_channel`; the socket is added to that channel's subscriber set after an ACL check against Postgres.
3. On a new message the server persists to the Message Store and broadcasts to all channel subscribers except the sender.
4. Typing, read receipts, pins, and presence are broadcast as lightweight events.
5. History is paginated from Cassandra by `channel_id` (keyset by `message_id`).

Data model

Cassandra: messages	chat history partitioned by channel
MongoDB	fallback message store
PostgreSQL: channels / memberships	channel metadata & access control

External dependencies

- Apache Cassandra / DataStax Astra DB
- MongoDB (fallback)

Failure modes & resilience

- **Cassandra unavailable** → store factory fails over to MongoDB.
- **Socket drop** → cleanup unsubscribes the client and broadcasts presence change.
- **Unauthorized channel access** → join rejected via Postgres ACL check.
- **Hot-path isolation** → chat volume never touches transactional Postgres tables.

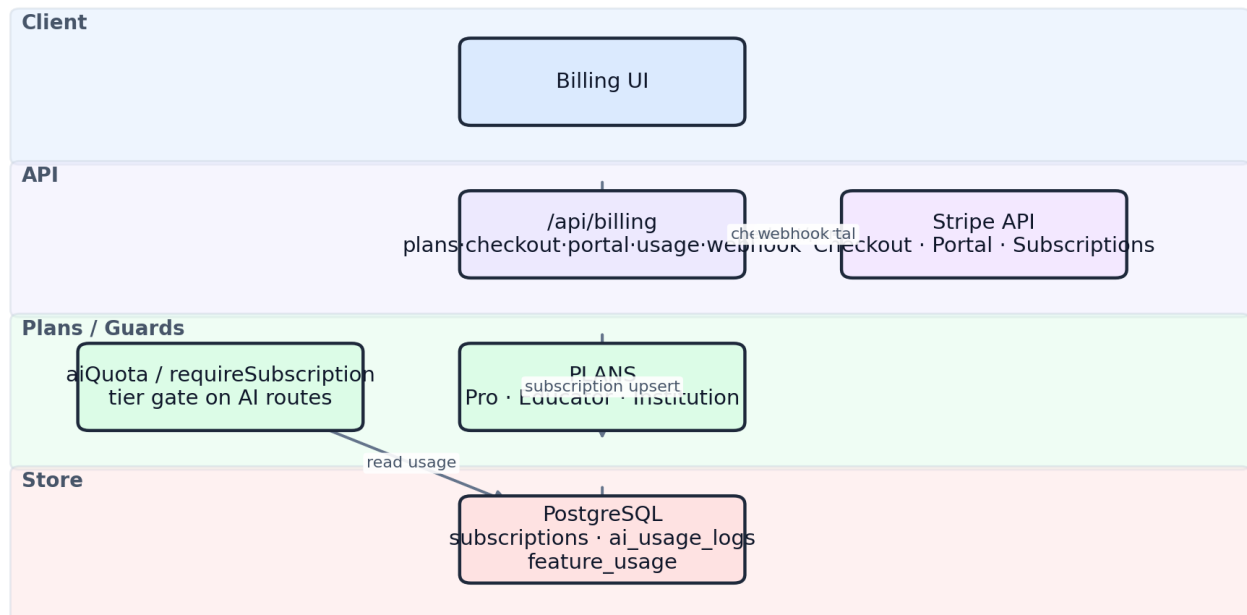
5. Billing & Quota

Purpose

Full Stripe billing lifecycle plus tiered AI usage limits. Plans gate access to paid features, while a quota guard caps monthly AI consumption per tier before any model spend occurs.

Architecture

Subsystem 5 — Billing & Quota (Stripe · Tiered AI limits)



Editable source: [diagrams/subsystems/05-billing-quota.excalidraw](#)

Key components

- **Billing Router** (`server/routes/billing.ts`) — `/plans`, `/subscription`, `/checkout`, `/portal`, `/usage`, `/webhook`.
- **Stripe integration** (`server/lib/stripe.ts`) — PLANS (Pro, Educator, Institution), checkout & customer-portal sessions, signature-verified webhooks.
- **Subscription guard** (`middleware/requireSubscription.ts`) — enforces a minimum tier on protected routes.
- **AI quota guard** (`middleware/aiQuota.ts`) — `checkAIQuota('ai_classroom' | 'ai_tutor' | 'ocr')` against monthly tier limits.

Request / data flow

1. Client fetches `/api/billing/plans` and starts `/checkout`; Stripe hosts payment.
2. Stripe posts a signature-verified `/webhook` (`checkout.session.completed`, subscription updates) which upserts the `subscriptions` row.
3. `/portal` returns a Stripe customer-portal link for plan changes & cancellation.
4. On each AI request, the quota guard reads usage and rejects over-limit calls before spend.
5. `/usage` exposes current consumption vs. the tier ceiling.

Data model

subscriptions	Stripe subscription state, period, tier
ai_usage_logs	per-call AI usage records
feature_usage	aggregated monthly counters

External dependencies

- Stripe (Checkout, Billing Portal, Subscriptions, Webhooks)

Failure modes & resilience

- **Forged webhook** → rejected unless the Stripe signature verifies.
- **Missed webhook** → subscription reconcilable from Stripe on next read.
- **Quota overrun** → guarded pre-spend; returns limit error, no model call.
- **Downgrade / cancel** → portal + webhook keep local tier in sync.

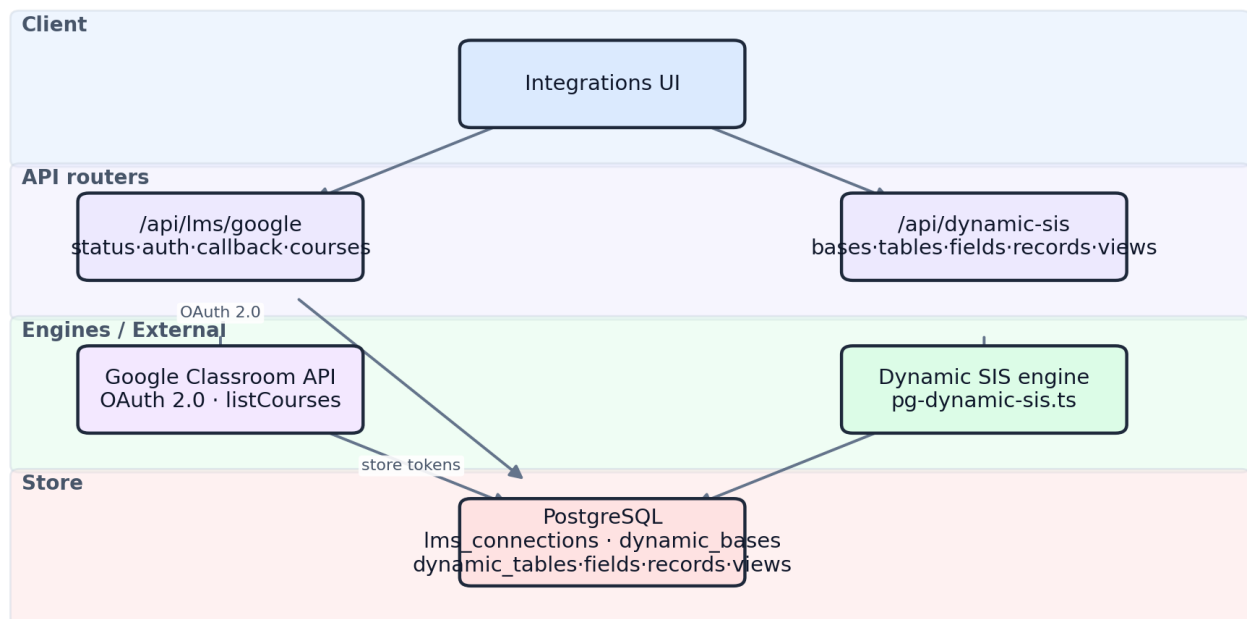
6. LMS & SIS Integrations

Purpose

Two workspace-scoped integration surfaces: Google Classroom import (courses & students) over OAuth, and a dynamic Student Information System — an Airtable-style engine of bases, tables, fields, records, and views that schools shape to their own data.

Architecture

Subsystem 6 — LMS & SIS Integrations (Google Classroom · Dynamic SIS)



Editable source: `diagrams/subsystems/06-lms-sis.excalidraw`

Key components

- **LMS Router** (`server/routes/lms.ts`) — `/google/status`, `/google/auth`, `/google/callback`, `/google/courses` with CSRF state.
- **Google Classroom client** (`server/lib/lms/googleClassroom.ts`) — OAuth token exchange + `listCourses`.
- **Dynamic SIS Router** (`server/routes/dynamic-sis.ts`) — CRUD for bases / tables / fields / records / views.
- **SIS engine** (`server/lib/pg-dynamic-sis.ts`) — all persistence, workspace-scoped by `workspace_id`.

Request / data flow

1. Teacher connects Classroom: `/google/auth` stores a state in session and redirects to Google.
2. `/google/callback` validates state, exchanges the code, and persists an `lms_connections` record with tokens.
3. `/google/courses` lists the connected account's courses for import.
4. SIS: clients CRUD bases → tables → fields → records; views define filtered/ordered projections.
5. Every SIS query is scoped to the caller's workspace, isolating tenant data.

Data model

lms_connections	per-user provider tokens & status
dynamic_bases / dynamic_tables	SIS containers
dynamic_fields / dynamic_records	schema & rows
dynamic_views	saved filtered/ordered views

External dependencies

- Google Classroom API (OAuth 2.0)

Failure modes & resilience

- **OAuth CSRF** → state mismatch aborts the callback.
- **Token expiry** → connection status surfaced via `/google/status` for re-auth.
- **Cross-tenant SIS access** → every engine query filtered by `workspace_id`.